APCS 程式設計系列 · U03 · 實作初級 · C++ / Python 對照版

迴圈 詳解

for 與 while · 累加計數與找最大最小 · 拆位數 · 巢狀迴圈與執行次數 · break / continue · 跑得夠快嗎

看這個單元的講解影片（YouTube）
【APCS C++ × Python】U03 迴圈｜for while、巢狀迴圈、break continue｜7 分鐘重點講解
錯比較多？回去看講解影片，手機掃描 QR code 或點這裡 https://youtu.be/mh9Nblr5LQg

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
B	C	A	D	B	A	C	D	B	C
11	**12**	**13**	**14**						
B	A	A	C						

Part A 程式識讀詳解

第 1 題 答案：(B)
- i 依序是 1、4、7、10（下一個 13 超過範圍）。
- 1 + 4 + 7 + 10 = 22。

第 2 題 答案：(C)
- i = 0 時內層跑 5 次，i = 1 跑 4 次……i = 4 跑 1 次。
- 5 + 4 + 3 + 2 + 1 = 15。

第 3 題 答案：(A)
- 每一圈取出 n 的個位數，接到 r 的尾巴，再把 n 的個位數去掉。
- r 依序是 5、54、543、5432、54321：數字被翻轉了。

第 4 題 答案：(D)
- 3、6 是 3 的倍數，被 continue 跳過；i = 8 時 8 > 7，break 跳出。
- 加到的是 1、2、4、5、7：總和 19。

【錯】i = 9 是 3 的倍數，但迴圈在 i = 8 已經結束了，輪不到它。

第 5 題 答案：(B)

- 注意 for (...) 後面直接接分號，迴圈本體是空的。
- i 依序 0、4、8、12，12 < 10 不成立才停，所以 i = 12。

【錯】以為迴圈最後一圈的 i（8）就是結束後的值。

第 6 題 答案：(A)

- range(0, 10, 4) 依序產生 0、4、8。
- 迴圈結束後，i 停在最後一個值 8（不會變成 12）。

【錯】C 題本的 for 結束後 i 會是 12，兩種題本不一樣。

第 7 題 答案：(C)

- 6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1。
- 箭頭數就是迴圈跑的圈數：8 圈。

第 8 題 答案：(D)

- 列出 (i, j)：i = 1 有 (1,1)；i = 2 有 (2,2)；i = 3 有 (3,1)(3,3)；i = 4 有 (4,2)(4,4)。
- i + j 是偶數的共 1 + 1 + 2 + 2 = 6 組。

第 9 題 答案：(B)

- mx 從 0 開始，三個數都比 0 小，mx 從來沒被更新。
- 所以印出 0，而不是真正的最大值 -2。

【錯】這就是「最大值初始值設成 0」的錯誤，題目常拿來考。

第 10 題 答案：(C)

- a 依序變成 3、9、27、81、243。
- a = 81 時 81 < 100 還會再跑一圈，變成 243 才停：k = 5。

第 11 題 答案：(B)

- 兩層各 n 次 = 10^{10} 次運算，遠超過每秒約 10^8 次。
- 單層 10^5 次、除以 2 約 17 次、公式 1 次，都很快。

第 12 題 答案：(A)

- do...while 是「先做再檢查」，所以至少會執行一次。
- 印出 10 之後 i = 11，11 < 5 不成立，結束。

【錯】當成一般 while（先檢查）就會選「沒有輸出」。

第 13 題 答案：(A)

- d = 3 時 9 % 3 == 0，印出 3 並 break。
- for...else 的 else 只有在「沒被 break」時才執行，所以不會印 prime。

第 14 題 答案：(C)

- 第一輪：9 + 8 + 7 + 5 = 29；29 >= 10，繼續。
- 第二輪：2 + 9 = 11；第三輪：1 + 1 = 2，小於 10 停止。

Part B 程式實作參考解答

實作 1 數列統計

解題思路

- 一邊讀一邊處理：總和累加、最大最小值隨時更新、偶數就計數。
- 最大最小值用第一個數當初始值（或 INT_MIN / INT_MAX）。

【錯】C++ 的總和最大 10^{14}，要用 long long。
【錯】最大值從 0 開始，全部是負數時會錯（範例 2）。
【錯】C++ 的負奇數 % 2 是 -1，判斷偶數寫 x % 2 == 0 最安全。

C++ 數列統計

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    long long sum = 0;
    int mx = INT_MIN, mn = INT_MAX, even = 0;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        sum += x;
        mx = max(mx, x);
        mn = min(mn, x);
        if (x % 2 == 0) even++;
    }
    cout << sum << " " << mx << " " << mn << " " << even << "\n";
}
```

Python

```python
import sys
data = sys.stdin.read().split()
n = int(data[0])
nums = list(map(int, data[1:1 + n]))
even = sum(1 for x in nums if x % 2 == 0)
print(sum(nums), max(nums), min(nums), even)
```

時間複雜度：O(n)

實作 2 數字翻轉

解題思路

- while (n > 0)：取出個位數 d = n % 10，加進數字和、接到翻轉數尾巴 rev = rev × 10 + d，再把 n 除以 10。
- 每跑一圈，位數加 1。

【錯】C++ 的 n 到 10^{18}，要用 long long。

【錯】用字串翻轉要記得去掉開頭的 0（1000 翻轉是 1，不是 0001）。

C++ 數字翻轉

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long digits = 0, sum = 0, rev = 0;
    while (n > 0) {
        int d = n % 10;
        sum += d;
        rev = rev * 10 + d;
        digits++;
        n /= 10;
    }
    cout << digits << " " << sum << " " << rev << "\n";
}
```

Python

```python
n = int(input())
digits = total = rev = 0
while n > 0:
    d = n % 10
    total += d
    rev = rev * 10 + d
    digits += 1
    n //= 10
print(digits, total, rev)
```

時間複雜度：O(位數) □ O(log n)

實作 3 3n+1 猜想

解題思路

- 不知道要跑幾次，用 while (n != 1)。
- 每操作一次，次數加 1，並且更新最大值。

【錯】過程中的數會衝到約 3×10^9（837799），C++ 的 n 和最大值都要用 long long。

【錯】n = 1 時迴圈一次都不跑，最大值要記得先設成 n 本身。

C++ 3n+1 猜想

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;
    long long steps = 0, mx = n;
    while (n != 1) {
        if (n % 2 == 0) n /= 2;
        else n = 3 * n + 1;
        steps++;
        mx = max(mx, n);
    }
    cout << steps << " " << mx << "\n";
}
```

Python

```python
n = int(input())
steps, mx = 0, n
while n != 1:
    if n % 2 == 0:
        n //= 2
    else:
        n = 3 * n + 1
    steps += 1
    mx = max(mx, n)
print(steps, mx)
```

時間複雜度：與操作次數成正比（題目範圍內最多 524 次）

實作 4 星號金字塔

解題思路

- 外層迴圈跑第 1 ~ n 層。
- 每一層裡面：先用一個迴圈印 n - i 個空白，再用一個迴圈印 2i - 1 個星號，最後換行。

【錯】空白數和星號數都跟 i 有關，先把 n = 3 的每一層列出來對一次公式，再寫程式。

C++ 星號金字塔

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int k = 0; k < n - i; k++) cout << ' ';
        for (int k = 0; k < 2 * i - 1; k++) cout << '*';
        cout << "\n";
    }
}
```

Python

```python
n = int(input())
for i in range(1, n + 1):
    print(" " * (n - i) + "*" * (2 * i - 1))
```

時間複雜度：$O(n^2)$（要印的字元數本來就是 n^2 等級）

實作 5 存錢計畫

解題思路

- 用 while 一天一天模擬：當天存的錢 = a + (天數 - 1) × d，累加到總額。
- 總額 >= M 就停，輸出天數。
- 最多大約 √(2M) ≈ 1.4×10^6 天，一層迴圈跑得完。

【錯】C++ 的 M 到 10^{12}，總額和每天存的錢都要用 long long。

【錯】「達到 M 以上」是 >=，寫成 > 會在剛好等於 M 時多算一天（範例 1：1+2+3+4 = 10，答案是 4）。

C++ 存錢計畫

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long m, a, d;
    cin >> m >> a >> d;
    long long total = 0, day = 0, today = a;
    while (total < m) {
        day++;
        total += today;
        today += d;
    }
    cout << day << "\n";
}
```

Python

```python
m, a, d = map(int, input().split())
total = day = 0
today = a
while total < m:
    day += 1
    total += today
    today += d
print(day)
```

時間複雜度：O(√M)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
for 迴圈範圍	A1、A5、A6		綠 黃 紅	講義第 1 段（差一錯誤）
while 與拆位數	A3、A7、A10、A14、B2、B3		綠 黃 紅	講義第 2 段
累加與最大最小	A9、B1、B5		綠 黃 紅	講義第 3 段
巢狀迴圈次數	A2、A8、B4		綠 黃 紅	講義第 4 段
break 與 continue	A4、A12、A13		綠 黃 紅	講義第 5 段
執行時間	A11		綠 黃 紅	講義第 6 段

覺得有幫助？幫老師一個忙

- **訂閱** YouTube「莊老數學教室」：youtube.com/@chuanglaomath
- 看到老師的影片，順手**按個讚**
- 把這份免費講義**分享給同學**：wade-chuang.github.io/jenfoo/apcs（不用註冊，直接下載）